EXPERIMENT 4 TCP

server.py file 1

```
import socket

HOST = '127.0.0.1'
PORT = 5000

questions = [
    "Q1: What is the OSI model purpose? (A: Standardize networking, B: Secure data, C: Speed
up networks)",
    "Q2: Which layer handles routing? (A: Transport, B: Network, C: Data Link)",
    "Q3: TCP is connection-oriented. True or False?"
]

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(1)

print(f"Server listening on port {PORT}...")
print("Waiting for client...")

conn, addr = server.accept()

student_name = conn.recv(1024).decode()
print(f"{student_name} connected from {addr}")

answers = []

for question in questions:
    conn.sendall(question.encode())

    answer = conn.recv(1024).decode()
    answers.append(answer)

conn.sendall("EXAM_COMPLETE".encode())
conn.close()

print("\n--- Final Exam Results ---")
print(f"Student: {student_name}")
for i, answer in enumerate(answers, 1):
    print(f"Q{i}: {answer}")

server.close()
```

client.py file 2 name

```
import socket

HOST = '127.0.0.1'
PORT = 5000

student_name = input("Enter your name: ")

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))

print(f"Connected to server at {HOST}:{PORT}")

client.sendall(student_name.encode())

while True:
    data = client.recv(1024).decode()

    if data == "EXAM_COMPLETE":
        print("\nExam submitted successfully!")
        break

    print("\n" + data)

    answer = input("Your answer: ")
    client.sendall(answer.encode())

client.close()
```

EXPERIMENT 5 UDP

server.py file name 1

```
import socket

HOST = '127.0.0.1'
PORT = 6000

positive_words = ["good", "great", "happy", "excellent",
                  "love", "amazing", "nice", "wonderful"]

negative_words = ["bad", "terrible", "sad", "hate",
                  "poor", "horrible", "angry", "disappointing"]

def analyze_sentiment(text):
    words = text.lower().split()

    pos_count = sum(1 for w in words if w in positive_words)
    neg_count = sum(1 for w in words if w in negative_words)

    if pos_count > neg_count:
        return "Positive"
    elif neg_count > pos_count:
        return "Negative"
    else:
        return "Neutral"

server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind((HOST, PORT))

print(f"UDP server listening on port {PORT}...")

while True:
    data, client_addr = server.recvfrom(1024)
    text = data.decode()

    if text.lower() == "exit":
        break

    print(f"Received from {client_addr}: {text}")
    sentiment = analyze_sentiment(text)
    print(f"Sentiment: {sentiment}")
    server.sendto(sentiment.encode(), client_addr)
server.close()
```

client.py file of 2 nd file

```
import socket

HOST = '127.0.0.1'
PORT = 6000

client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

while True:
    text = input("Enter text (or 'exit' to quit): ")

    client.sendto(text.encode(), (HOST, PORT))

    if text.lower() == "exit":
        break

    data, addr = client.recvfrom(1024)
    print("Sentiment:", data.decode())

client.close()
```

EXPERIMENT 7 OSPF

```
Enable
Conf t
router ospf 1
network 192.168.4.0 0.0.0.255 area 1
network 13.0.0.0 0.255.255.255 area 1
network 12.0.0.0 0.255.255.255 area 1

enable
show ip interface brief
show ip ospf neighbor
show ip route
show ip protocols

ping 192.168.3.2
ping 192.168.4.2
```

EXPERIMENT 8-ACL

```
enable
configure terminal
access-list 101 deny tcp 140.80.24.0 0.0.7.255 host 140.80.120.7 eq 80
access-list 101 deny tcp 140.80.24.0 0.0.7.255 host 140.80.120.7 eq 443
access-list 101 permit ip any any
interface gigabitEthernet0/0
ip access-group 101 in
end
show access-lists
copy running-config startup-config
```

—----------

```
enable
configure terminal
access-list 101 permit tcp host 192.168.1.2 host 10.0.0.10 eq 80
access-list 101 permit tcp host 192.168.1.2 host 10.0.0.10 eq 443
access-list 101 deny ip host 192.168.1.2 host 10.0.0.10
access-list 101 permit ip any any

interface gigabitEthernet0/0
ip access-group 101 in
end
```

Service	Protocol	Port
HTTP	TCP	80
HTTPS	TCP	443
FTP — Data	TCP	20

FTP — Control	TCP	21
SSH	TCP	22
Telnet	TCP	23
SMTP — Email sending	TCP	25
DNS	TCP/UDP	53
DHCP — Server	UDP	67
DHCP — Client	UDP	68
POP3	TCP	110
NTP — Time synchronization	UDP	123
IMAP — Email	TCP	143

MININET

```
from mininet.net import Mininet
from mininet.node import OVSBridge
from mininet.cli import CLI

net = Mininet(controller=None, switch=OVSBridge)

# PASTE YOUR TOPOLOGY CODE HERE

net.start()
net.pingAll()
CLI(net)
net.stop()
```

1. Star Topology

```
s1 = net.addSwitch('s1')

for i in range(1, 5):
    h = net.addHost('h' + str(i))
    net.addLink(h, s1)
```

2. Linear Topology

```
s1 = net.addSwitch('s1')
s2 = net.addSwitch('s2')
s3 = net.addSwitch('s3')

net.addLink(s1, s2)
net.addLink(s2, s3)

for i, s in enumerate([s1, s2, s3], 1):
    h = net.addHost('h' + str(i))
    net.addLink(h, s)
```

3. Tree Topology

```
s1 = net.addSwitch('s1')
s2 = net.addSwitch('s2')
s3 = net.addSwitch('s3')

net.addLink(s1, s2)
net.addLink(s1, s3)

for i in range(1, 5):
    h = net.addHost('h' + str(i))
    net.addLink(h, s2 if i <= 2 else s3)
```

4. Ring Topology

```
switches = [net.addSwitch('s'+str(i)) for i in range(1, 5)]

for i in range(4):
    net.addLink(switches[i], switches[(i+1) % 4])

    h = net.addHost('h'+str(i+1))
    net.addLink(h, switches[i])
```

5. Mesh Topology

```
s1 = net.addSwitch('s1', stp=True)
s2 = net.addSwitch('s2', stp=True)
```

```
s3 = net.addSwitch('s3', stp=True)

net.addLink(s1, s2)
net.addLink(s2, s3)
net.addLink(s1, s3)

for i, s in enumerate([s1, s2, s3], 1):
    h = net.addHost('h' + str(i))
    net.addLink(h, s)
```

6. Bus Topology

```
s1 = net.addSwitch('s1')

for i in range(1, 5):
    h = net.addHost('h' + str(i))
    net.addLink(h, s1)
```

Command

Purpose

`nodes`

Displays all nodes.

`net`

Displays links between nodes.

`pingall`

Tests connectivity between all hosts.

`h1 ping -c 3 h3`

Sends three ping requests from H1 to H3.

`iperf h1 h2`

Measures TCP bandwidth between H1 and H2.

`dump`

Displays node details, including interfaces and addresses.

`exit`

Exits the Mininet CLI.

MININET WITH ROUTER

```
from mininet.net import Mininet
from mininet.node import Host
from mininet.cli import CLI
from mininet.log import setLogLevel


class LinuxRouter(Host):
    def config(self, **params):
        super().config(**params)
        self.cmd('sysctl -w net.ipv4.ip_forward=1')

    def terminate(self):
        self.cmd('sysctl -w net.ipv4.ip_forward=0')
        super().terminate()


def routerTopology():
    net = Mininet(controller=None)

    # Create switches
    s1 = net.addSwitch('s1')
    s2 = net.addSwitch('s2')

    # Create router
    r1 = net.addHost('r1', cls=LinuxRouter)

    # Create hosts
    h1 = net.addHost(
        'h1', ip='10.0.1.1/24',
        defaultRoute='via 10.0.1.254'
    )
    h2 = net.addHost(
        'h2', ip='10.0.1.2/24',
        defaultRoute='via 10.0.1.254'
    )
    h3 = net.addHost(
        'h3', ip='10.0.2.1/24',
        defaultRoute='via 10.0.2.254'
```

```
    )
    h4 = net.addHost(
        'h4', ip='10.0.2.2/24',
        defaultRoute='via 10.0.2.254'
    )

    # Connect hosts to switches
    net.addLink(h1, s1)
    net.addLink(h2, s1)
    net.addLink(h3, s2)
    net.addLink(h4, s2)

    # Connect router to both networks
    net.addLink(
        s1, r1,
        intfName2='r1-eth0',
        params2={'ip': '10.0.1.254/24'}
    )
    net.addLink(
        r1, s2,
        intfName1='r1-eth1',
        params1={'ip': '10.0.2.254/24'}
    )

    # Start and test network
    net.start()
    net.pingAll()

    CLI(net)
    net.stop()


if __name__ == '__main__':
    setLogLevel('info')
    routerTopology()
```